Day 5: Endpoint Baseline Collection Report

Establishing a Security Baseline for Future Alert Investigation

Field	Details
Intern Name	Amrita Pathak
Date	07 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	11:59 PM IST

Contents

1. Objective 3
2. Tools and Environment 3
3. Key Terms 3
4. Why This Task Matters 3
5. Hostname and Current User 4
6. Running Processes 5
7. Active Network Connections 6
8. Startup Items and Services 8
9. Verifying the Baseline Was Saved Correctly 11
10. Findings 13
    10.1 The network baseline is the strongest evidence in this collection 13
    10.2 The distinction between running and enabled services closes a real persistence blind spot 13
    10.3 The corrected command error is itself useful evidence of process integrity 14
    10.4 Kernel threads and user processes need to be told apart before a comparison is useful 14
11. Conclusion 14

1. Objective

Collect a basic security baseline from a lab endpoint: hostname, current user, running processes, active network connections, and startup/services. Save the actual command output, not just screenshots, so it can be directly compared against future output during a real alert investigation.

2. Tools and Environment

Item	Detail
Endpoint baselined	Kali Linux 2026.2
Hostname / user tools	hostname, whoami, id
Process listing tool	ps
Network connection tool	ss
Service management tool	systemctl
Evidence folder	~/day5_baseline

Kali was chosen over the lab's Windows 11 VM for this baseline. The task explicitly treats Windows and Linux as equally valid targets, and Kali offered a faster, more reliable environment to work in without sacrificing anything the task requires. Every command's output was redirected into a saved text file rather than only captured as a screenshot, since the entire value of a baseline is its ability to be directly compared, line by line, against future output, something only a saved file supports.

3. Key Terms

- **Baseline**: a documented snapshot of what is normal on a system before any incident occurs. Its value comes entirely from being a comparison point; without one, an analyst has no way to tell if something found during an investigation is new or has always been there.
- **PID (Process ID):** a unique number the operating system assigns to each running process, used to identify and reference it.
- **Kernel thread**: a process belonging to the operating system's own core rather than a user application, shown by ps inside square brackets, for example [kworker/0:1].
- **Listening port**: a port a service has opened to wait for incoming connections, captured with ss -l. A port listed here is a potential entry point into the system.
- **Established connection**: an active, already-open connection between this machine and another, as opposed to a port merely waiting for one.
- **Ephemeral port**: a temporary, high-numbered port automatically assigned to the client side of an outgoing connection, as opposed to the fixed, well-known port a server listens on.
- **systemd unit / service**: a background program managed by systemd, Kali's service manager. A service can be running right now, enabled to start automatically on boot, or both, and these are not always the same list.
- **Persistence**: a technique used to survive a reboot. A backdoor that only exists in memory disappears when the machine restarts, so attackers who want to remain often register a service or startup entry, which is exactly why startup/service data belongs in a baseline.

4. Why This Task Matters

A baseline is only worth collecting if the reason for each piece of it is understood, not just the commands used to gather it. This section exists so this report can be revisited later, after the specific commands are forgotten, and still explain why they were run in the first place.

The core problem a baseline solves is this: a SOC analyst investigating an alert almost never knows, on its own, whether something is new. An alert might say "unusual process detected" or "suspicious network connection." Unusual compared to what? Suspicious compared to what? Without a documented answer to that question captured before anything happened, an analyst is left guessing, or worse, wasting investigation time chasing something that has actually been normal on this system the whole time. A baseline turns that guess into a fact: this process was or was not present before, this port was or was not open before, this service was or was not configured to start before.

Each of the four categories collected here defend against a different stage of a real compromise, which is why all four are required rather than any one being sufficient on its own.

- **Hostname and current user establish identity**. Every log, every alert, every piece of telemetry a SOC tool ever produces is tagged with a hostname and a user. If either of those baseline facts ever silently changes, for example a machine being renamed to blend in, or a new administrative account appearing, that is one of the simplest and most reliable signs something is wrong, but only if the original values were documented first.

- **Running processes defend against the execution stage of an attack**. Malware, backdoors, and cryptominers all have to run as a process to do anything at all. A process baseline is what turns "there's a process running" from a meaningless observation into "this process was not here before," which is an actionable finding.

- **Active network connections defend against the stage** where an attacker's tools try to communicate outward, commonly called command-and-control. Attackers deliberately give malicious processes boring, forgettable names specifically so that a process list alone won't catch them. A network connection to an unfamiliar address is often the very first thing that actually gives a well-disguised compromise away, but only if the analyst knows what connections were normal beforehand.

- **Startup items and services defend against persistence**, the stage where an attacker tries to survive a reboot. A compromise that only exists in running memory is temporary and often disappears the moment the machine restarts. An attacker who wants to stay has to register something that starts automatically, which is exactly what this category is built to catch if it ever changes.

Put together, these four categories are not four unrelated checklist items. They map to four different points in how a real compromise actually unfolds: get in, run something, talk to an outside server, and try to survive a reboot. A baseline that only covered one or two of these would leave real gaps. This is also why the task specifically asks for saved command output rather than just screenshots: a future investigation depends on being able to directly compare new output against this exact saved evidence, line by line, not on remembering what a screenshot looked like.

5. Hostname and Current User

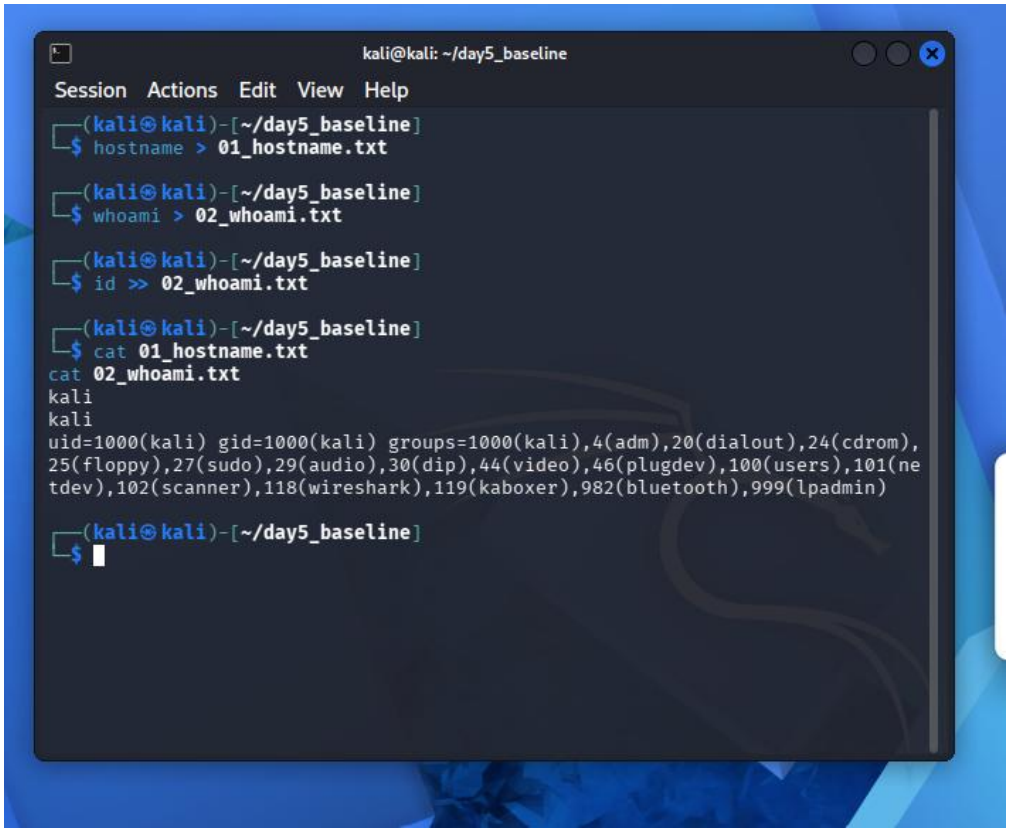
The first two commands establish the most basic facts an analyst needs before anything else: what machine this is, and who is legitimately using it.

```
hostname > 01_hostname.txt
whoami > 02_whoami.txt
id >> 02_whoami.txt
```

Each command was chosen for a distinct reason rather than picked arbitrarily. hostname reads the system name the kernel itself has stored, the same value every log source on this machine stamps onto its output, so it is the definitive answer to "what machine is this." whoami resolves the currently logged-in session back to a username, answering "who is running commands right now." id goes a level deeper than whoami by also listing every group that user belongs to, not just their name, which matters because group membership, not just identity, is what actually determines what a user is allowed to do on this system.

The first attempt at this combined all three commands onto a single line separated by && without line breaks, which caused hostname to interpret the word "whoami" as an argument telling it to rename the machine, rather than as a separate

command. This was caught immediately from the resulting error ("you must be root to change the host name") and corrected by running each command on its own line.



Corrected output: hostname is kali, the logged-in user is kali (UID 1000), and group membership includes sudo, confirming this account holds administrative privileges.

Reading this output field by field: UID 1000 and GID 1000 are the numeric identifiers Linux actually uses internally, kali is simply the human-readable label attached to them. The groups list that follows is the important part for a baseline, since it is a complete record of every privilege boundary this account sits inside.

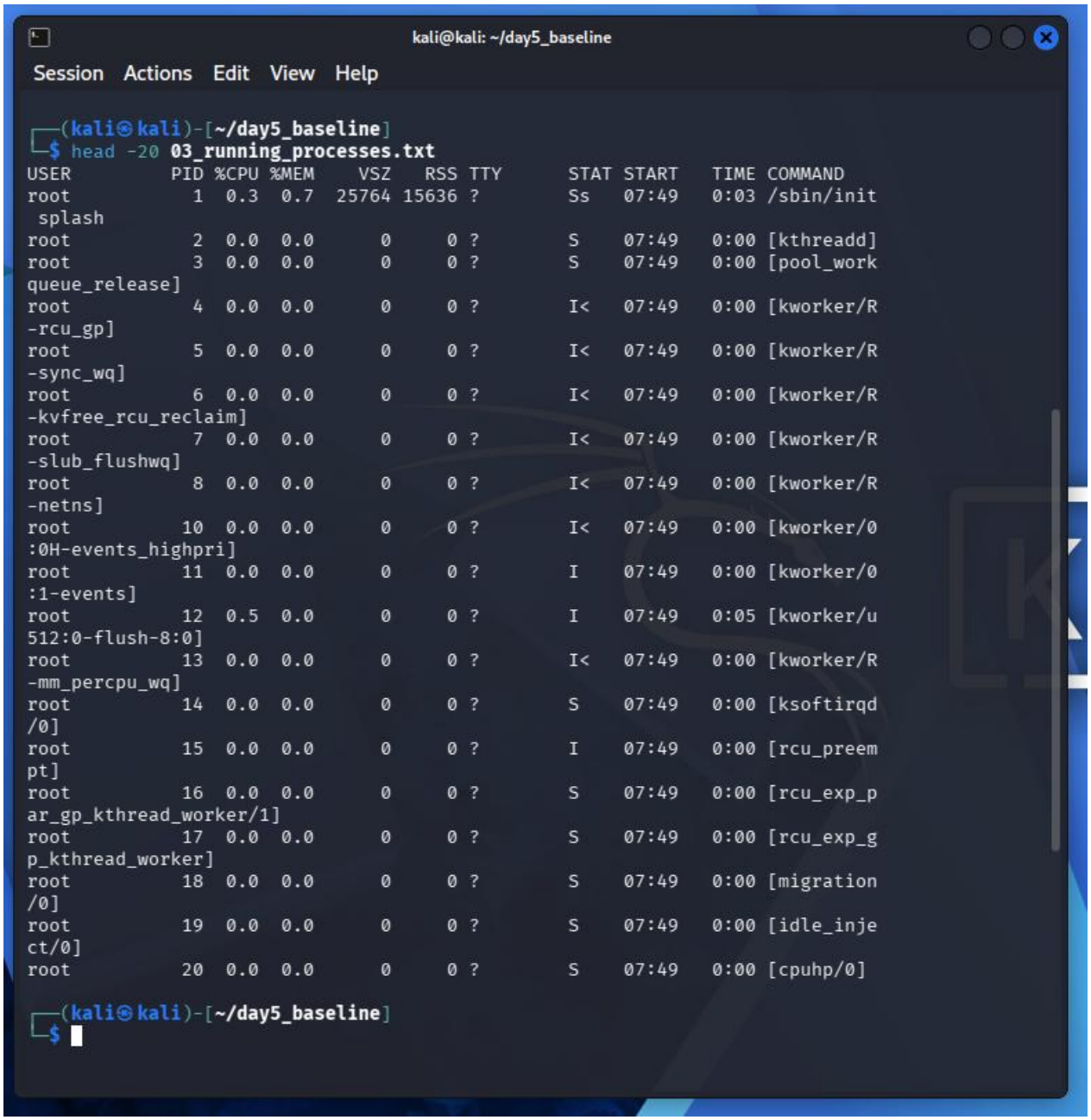
sudo stands out specifically because it grants the ability to escalate to root, meaning this account can execute commands as the system's most privileged user after authenticating. wireshark also appears, consistent with this being the same account used for packet capture in earlier tasks. The sudo group membership is worth stating explicitly as a baseline fact: if a future investigation ever showed root-level actions coming from an account that was not in this group at baseline, that would be a meaningful discrepancy worth immediate attention, since it would mean either the group membership changed or a different escalation path was used.

6. Running Processes

A full process listing captures every program running on the system at baseline, which user owns it, its resource usage, and the exact command that launched it.

```
ps aux > 03_running_processes.txt
```

The specific flag combination matters here. a tells ps to show processes belonging to every user on the system, not only the current one. u switches the output into the user-oriented format that includes the owning user, %CPU, and %MEM columns rather than a bare process list. x includes processes that are not attached to any terminal at all, which covers background daemons and services, exactly the category most likely to be relevant in a security investigation. Leaving out x in particular would have silently hidden a large share of what actually needed to be on this baseline.



A portion of the process baseline. Every entry shown is owned by root, in a sleeping or idle state, with square-bracketed names indicating kernel threads rather than user applications.

Reading the columns left to right: USER shows root for every process in this excerpt, PID is the unique process identifier the kernel assigned, %CPU and %MEM sit near 0.0 across the board since this is a freshly booted system with nothing under load, and STAT shows the process's current state, S for sleeping (idle, waiting to be woken up) and I for an idle kernel thread, neither of which is a state worth any concern.

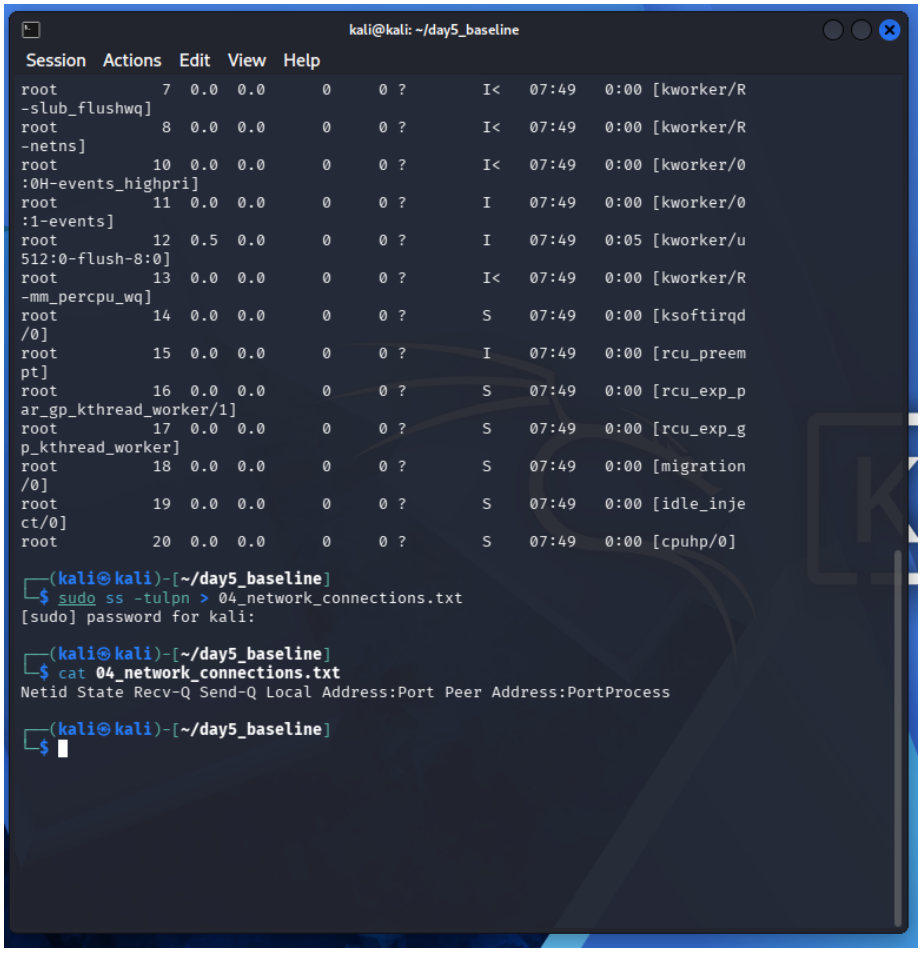
The square-bracket notation in the COMMAND column matters most for reading this list correctly going forward: entries like [kworker/R-rcu_gp] or [ksoftirqd/0] are kernel worker threads, core parts of the Linux kernel itself, not separate applications someone installed. A real user-facing program, a browser, a terminal, a server process, would appear without brackets and typically be owned by a non-root user. Recognizing this distinction is what allows an analyst to quickly separate normal system infrastructure from something that actually warrants a closer look during a future comparison.

7. Active Network Connections

Two passes were needed to capture this category completely, since a single ss command cannot cleanly show both listening ports and already-established connections in the way this baseline needed. The first command targets listening ports specifically.

```
sudo ss -tulpn > 04_network_connections.txt
```

Breaking down every flag: t restricts the output to TCP sockets, u adds UDP sockets alongside them, l restricts results to listening sockets only, meaning ports actively waiting for an incoming connection rather than ones already in use, p requests the owning process name and PID for each socket (this is why sudo is required, a normal user cannot see which process owns a socket that belongs to another user), and n prints raw port numbers instead of resolving them to service names, which is both faster and avoids any ambiguity from a resolved name being misleading.



Zero listening ports returned. The column headers print, but no services on this machine are currently waiting to accept incoming connections.

This is a genuinely meaningful baseline finding, not an empty or failed result. It means no SSH daemon, no web server, and no other service is currently exposed and waiting for a connection on this machine. This is exactly the kind of fact worth having on record: if a future investigation ever showed a new listening port that was not present here, that absence at baseline makes the new port immediately suspicious rather than ambiguous.

Since the task asks for active connections broadly, not just listening ports, a second command was run to also capture any already-established sessions.

```
sudo ss -tunap >> 04_network_connections.txt
```

This second command keeps t, u, n, and p from before but replaces l with a, which switches the filter from "listening only" to "all sockets," including connections that are already fully established between this machine and another. This is the command that actually satisfies the task's request for active connections, since the first command alone would have reported a clean result even if this machine had live, ongoing connections to somewhere else.



One established connection found: local port 68 to remote 192.168.84.254 port 67, owned by NetworkManager, using UDP.

Ports 67 and 68 are a specific, well-known pair reserved for DHCP, port 67 is the server side, port 68 is the client side. This single connection is Kali's own NetworkManager maintaining its IP address lease with the VMware virtual DHCP service, the same address seen acting as the network gateway during the Day 3 host discovery sweep.

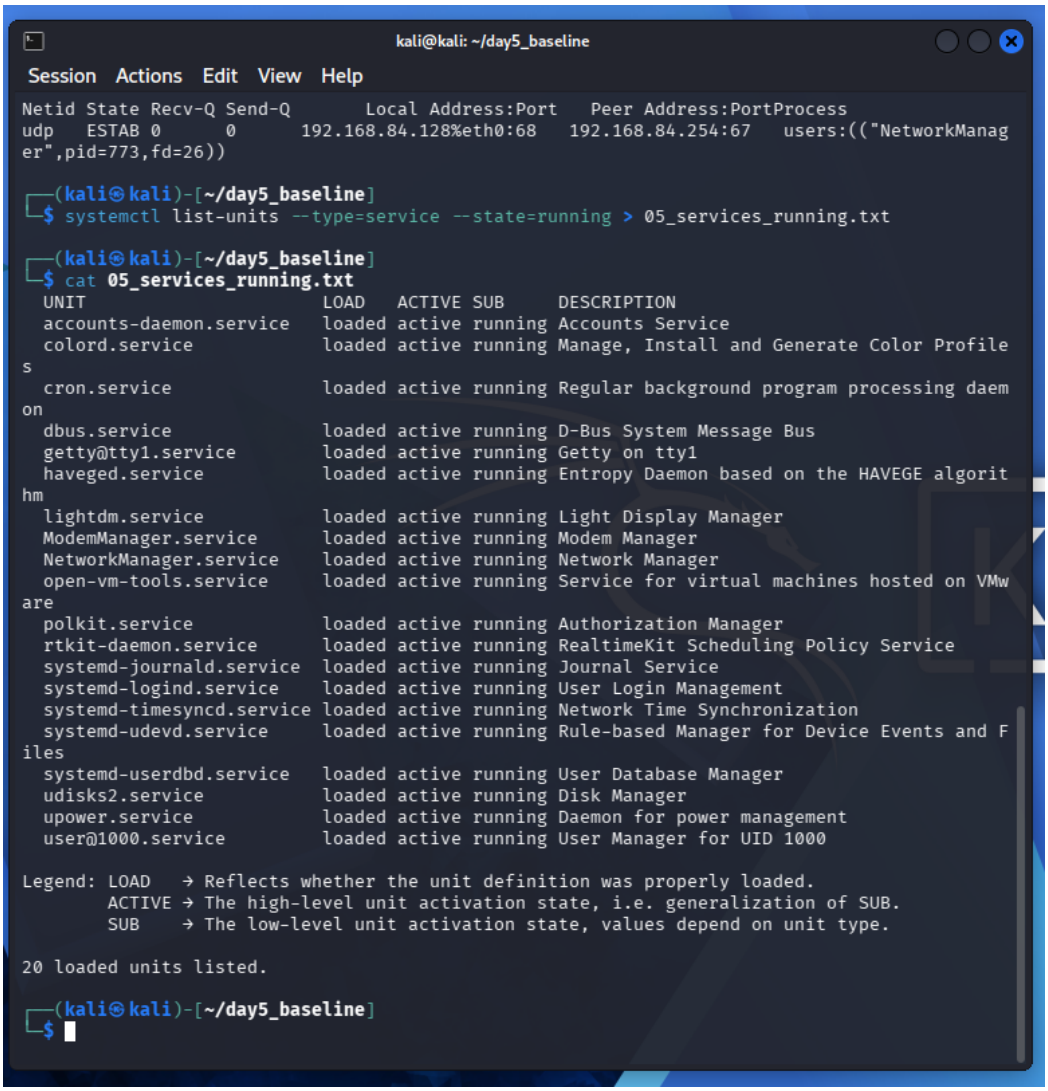
This is ordinary, expected background behavior for any machine on a DHCP-assigned network, not a finding requiring escalation. Taken together, the network baseline for this machine is about as clean as it gets: zero exposed listening services and exactly one legitimate, explainable connection.

8. Startup Items and Services

This category has two parts that are not always identical: what is running right now, and what is configured to start automatically on the next boot. Capturing both matters because a service could be active now without being set to survive a reboot, or the reverse.

```
systemctl list-units --type=service --state=running > 05_services_running.txt
```

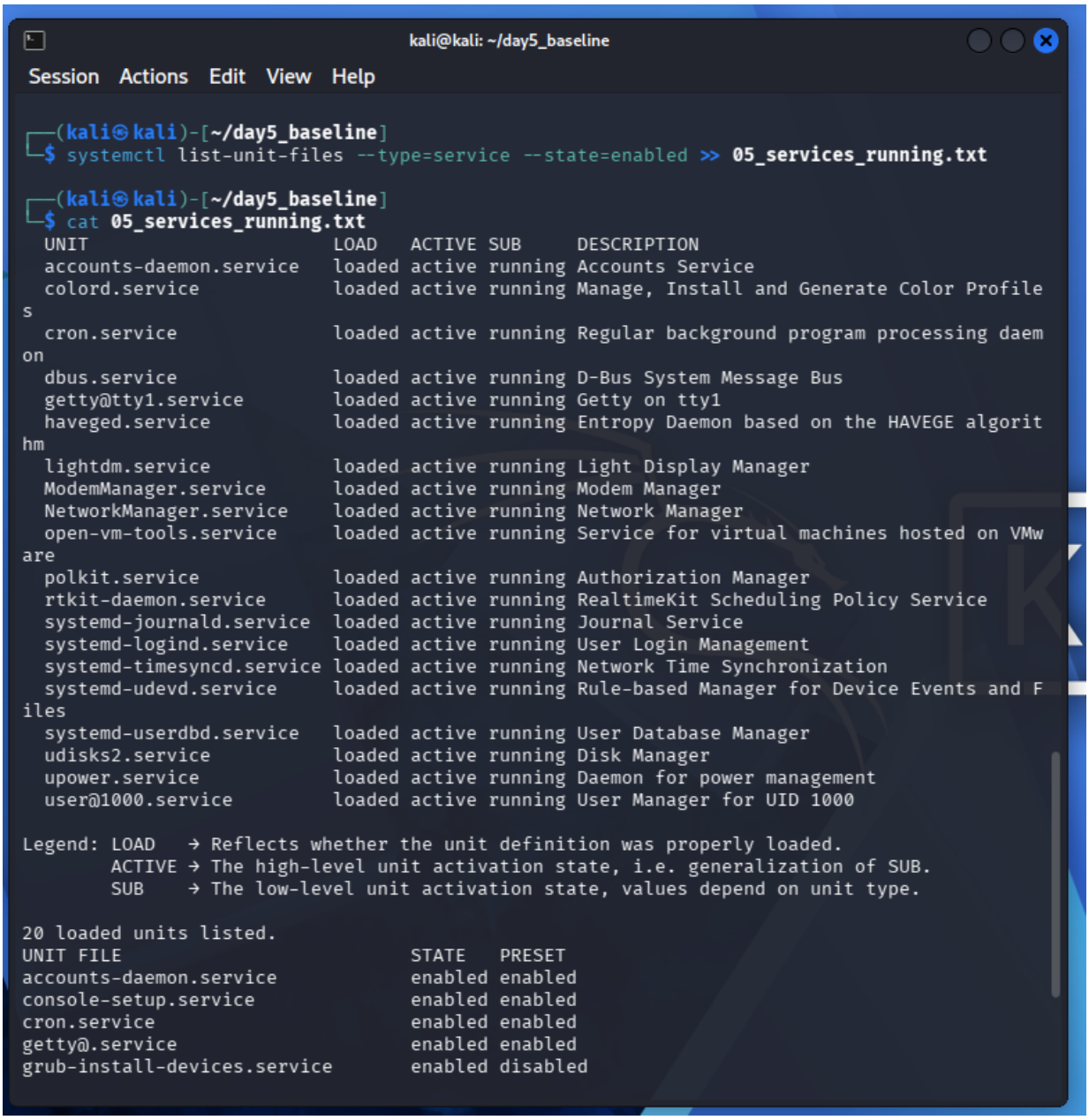
list-units queries systemd for units it currently has loaded into memory, filtered here to --type=service (excluding other unit types like mounts or timers, which are not relevant to this baseline) and --state=running (excluding services that are loaded but currently inactive). This answers "what is actually active on this system right now."



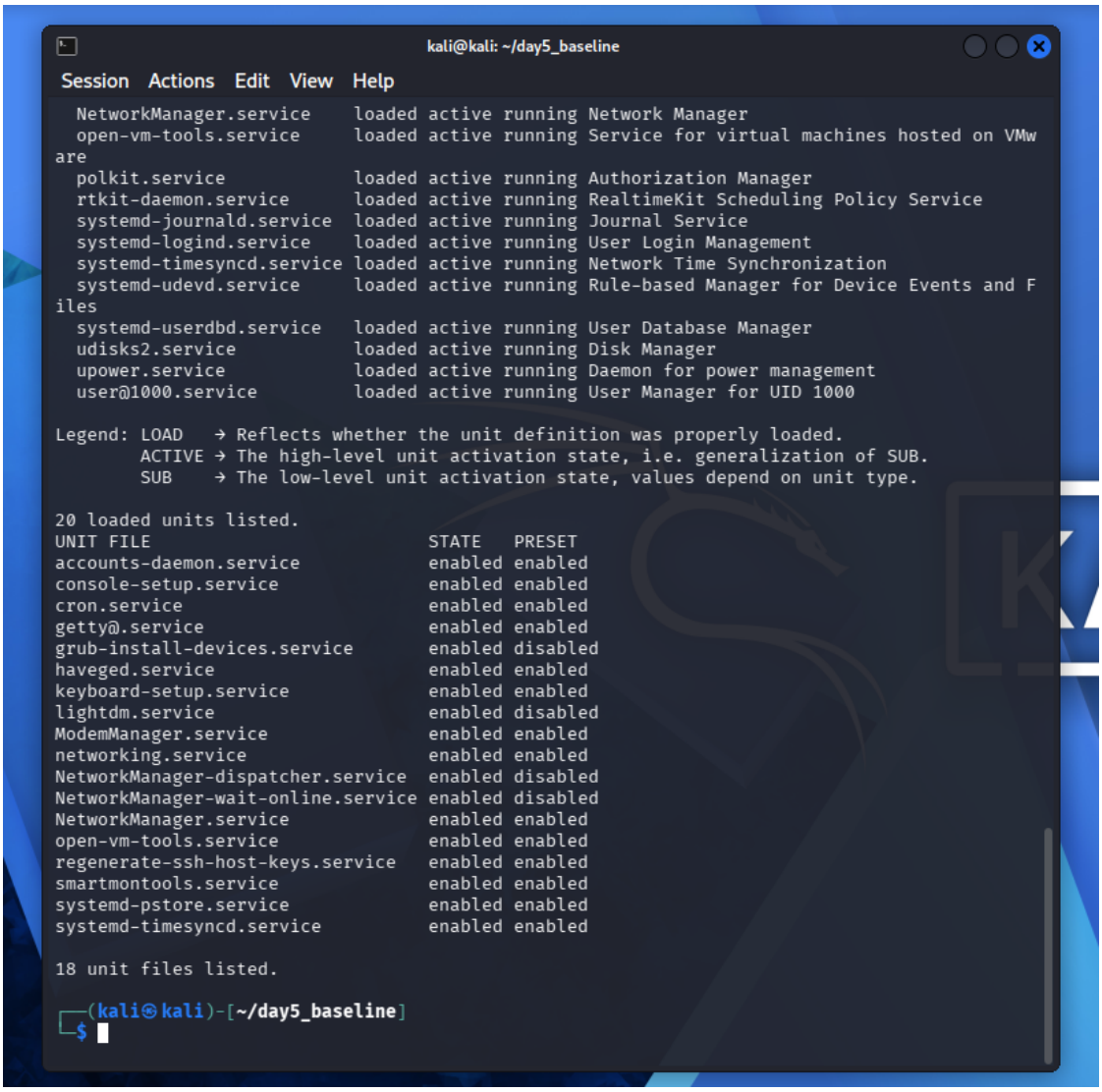
20 services currently loaded and running, all standard Kali/Debian infrastructure: display manager, network manager, cron, dbus, journal logging, and similar core services.

```
systemctl list-unit-files --type=service --state=enabled >> 05_services_running.txt
```

This second command is deliberately different from the first, not a repeat of it. list-unit-files reads the actual service definition files stored on disk and reports their configured enablement state, regardless of whether that service happens to be running at this exact moment. This distinction is exactly what makes it possible to catch a persistence mechanism a future attacker might install: a malicious service could be enabled to start on the next boot without necessarily being active yet, which the first command alone would never reveal.



Services enabled to start automatically on boot, continued from the running-services list above.



18 unit files confirmed enabled in total, including NetworkManager, cron, and SSH host key regeneration, none of which are unexpected on this system.

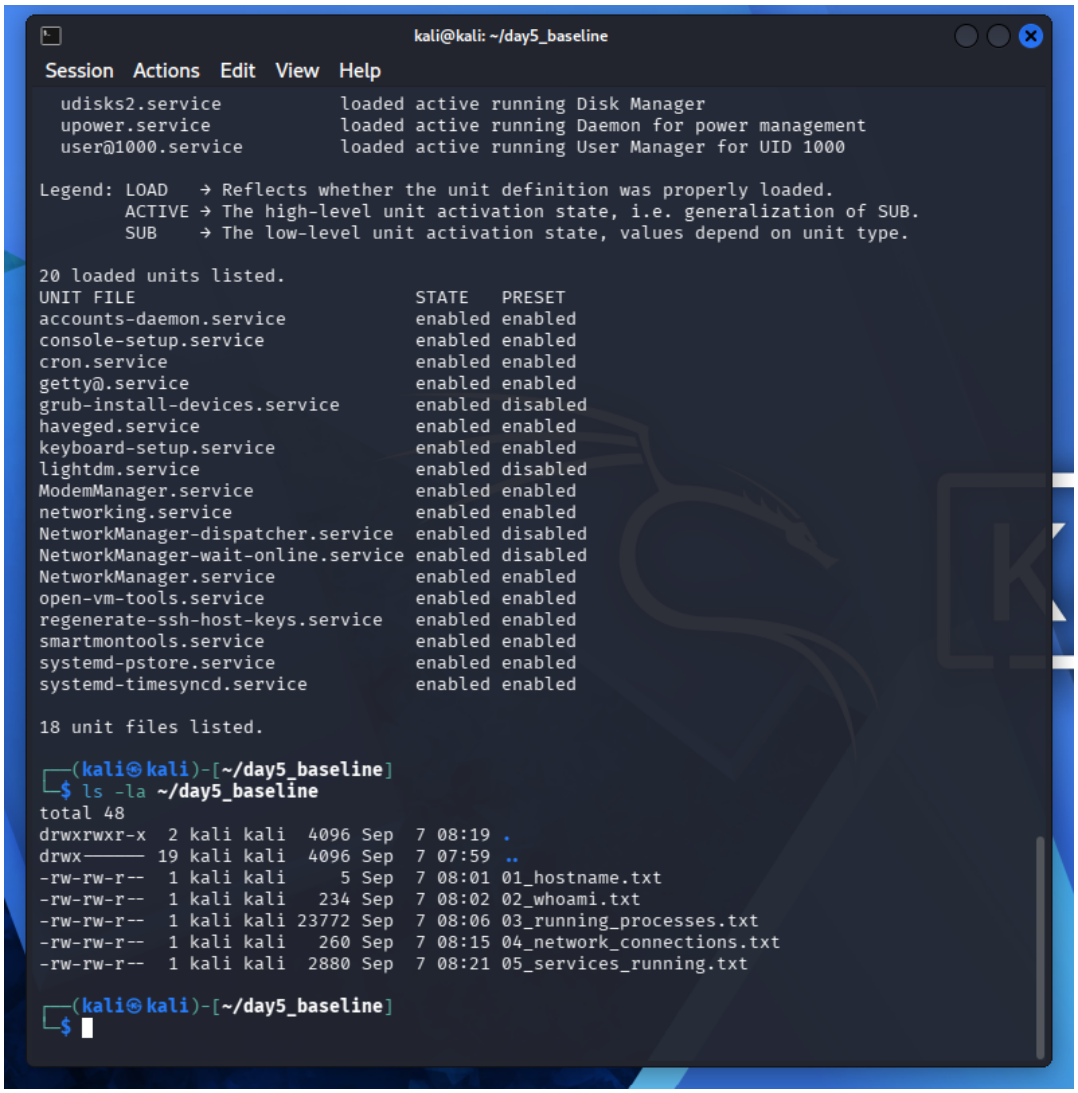
20 running versus 18 enabled is not a discrepancy worth flagging, it reflects that a small number of services (such as per-session login handlers) start on demand rather than existing as a single persistent enabled unit. Every service name across both lists is standard, expected Kali infrastructure. Nothing here indicates a service was added outside of the normal installation, which is exactly the conclusion this category of baseline is meant to support.

9. Verifying the Baseline Was Saved Correctly

Before concluding, the evidence folder was checked to confirm every file was actually written with real content, since the first hostname/whoami attempt had silently produced two empty files after the command error. This step exists for the same reason evidence handling matters in any investigation: a file that exists but is empty is not evidence, and the only way to know the difference is to actually check, not assume.

```
ls -la ~/day5_baseline
```

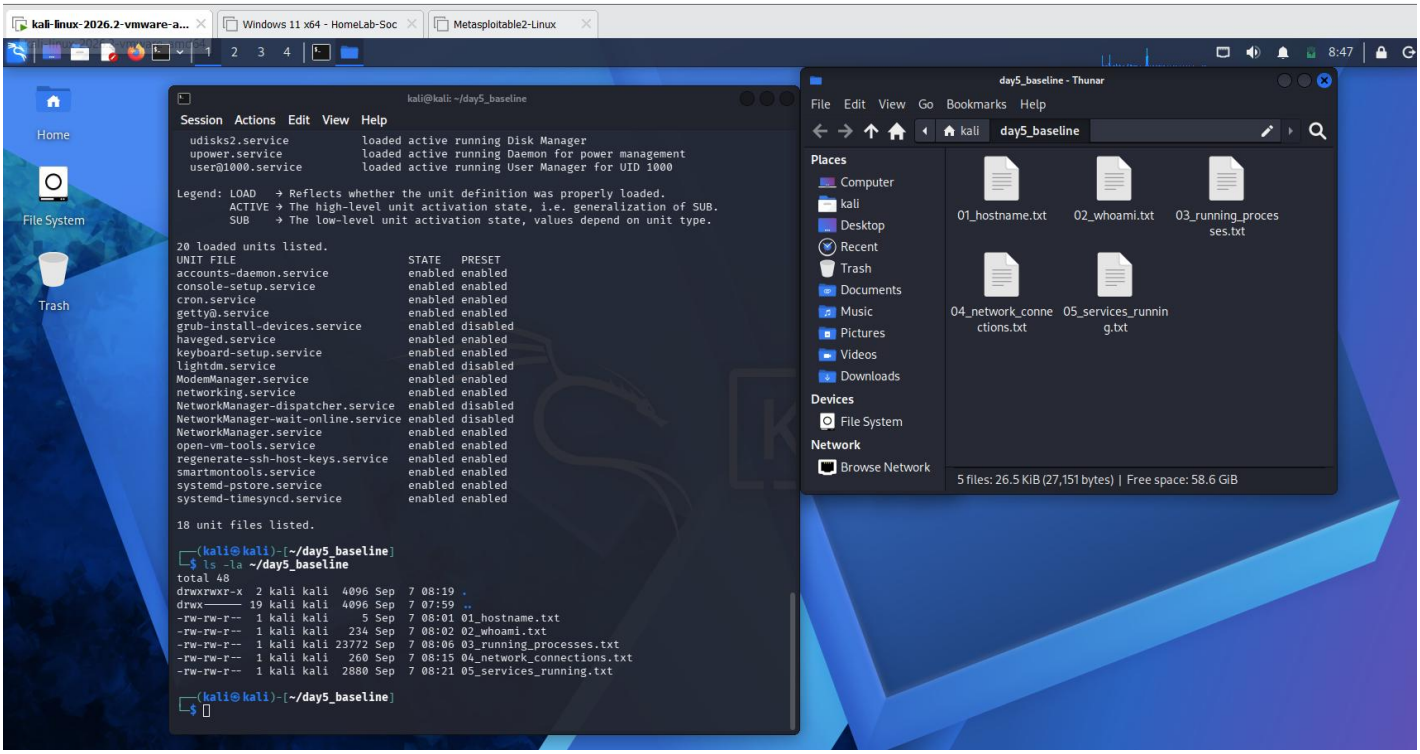
The two flags here are both deliberate. l switches to the long listing format, which is what actually exposes file size, ownership, permissions, and modification time, a bare ls would only have shown filenames, hiding exactly the detail (file size) needed to catch the earlier empty-file problem. a includes hidden files (anything starting with a dot), which matters for completeness even though none turned out to be present in this particular folder.



All five evidence files present with non-zero file sizes, confirming the baseline was captured and saved correctly.

Reading this output field by field: -rw-rw-r-- is the file's permission string, meaning the owner and the owning group can both read and write the file, while everyone else on the system can only read it, appropriate for evidence that should not be casually altered by other accounts. The number immediately after that (1 for each of these files) is the link count, a filesystem-level detail not relevant to this baseline. The two name fields that follow are the owner and group, kali and kali in every case here, then the file size in bytes, then the modification date and time, and finally the filename itself. The size column is the one that actually matters most for this task, since it is what proved the difference between the earlier broken attempt (0 bytes) and this corrected one.

As a second, independent confirmation that this evidence genuinely lives where it is supposed to, the same folder was also opened in Thunar, Kali's graphical file manager, alongside the terminal.



Thunar's breadcrumb path (kali > day5_baseline) confirms the folder sits directly inside the kali user's home directory, matching the terminal's own ls -la output for the same location side by side.

This second screenshot is not strictly required, since the terminal prompt itself (kali@kali:~/day5_baseline) and the ls -la output already prove the files exist at that path, ~ always expands to the current user's home directory, which for the kali account is /home/kali. Including a graphical confirmation alongside the command-line one is still worthwhile for a report like this one, though, since it gives a reader who is less comfortable reading a terminal prompt an equally clear, independent way to see that the evidence sits exactly where it should.

File	Size	Contents
01_hostname.txt	5 bytes	Machine hostname
02_whoami.txt	234 bytes	Current user and full identity/group membership
03_running_processes.txt	23,772 bytes	Full process listing (ps aux)
04_network_connections.txt	260 bytes	Listening ports and active connections
05_services_running.txt	2,880 bytes	Running services and boot-enabled services

10. Findings

10.1 The network baseline is the strongest evidence in this collection

A machine with zero listening services and exactly one explainable DHCP connection is a genuinely useful reference point. Almost any future finding here, a new listening port, a connection to an unfamiliar remote address, a process suddenly communicating over the network, would stand out immediately against this baseline precisely because there is so little noise to filter through first.

10.2 The distinction between running and enabled services closes a real persistence blind spot

Checking only currently running services would have missed anything an attacker configured to start on the next boot but had not yet triggered. Capturing both the running list and the enabled list means a future investigation can catch persistence mechanisms even if the malicious service has not been started yet, not just ones already active.

10.3 The corrected command error is itself useful evidence of process integrity

The first attempt to capture hostname and user data failed due to a command syntax error (three commands run together without line separation caused hostname to misinterpret whoami as an argument), producing two empty files. This was caught by checking the file sizes in a file manager before proceeding, and corrected by re-running each command on its own line. Documenting this here matters for two reasons: it shows the verification step (checking that saved evidence actually contains data, not just that a file exists) is a necessary part of collecting a baseline, not an optional extra, and it confirms the final saved files are genuinely correct rather than assumed to be.

10.4 Kernel threads and user processes need to be told apart before a comparison is useful

A large share of the process baseline consists of kernel worker threads, identifiable by their square-bracket naming and root ownership, which exist as core parts of the operating system rather than installed applications. An analyst comparing a future process list against this baseline needs to recognize this pattern to avoid wasting investigation time on infrastructure that is expected to always be present, and instead focus attention on new, non-bracketed, user-space processes.

11. Conclusion

A complete endpoint baseline was collected across all four required categories, hostname and current user, running processes, active network connections, and startup/services, with every command's actual output saved to disk rather than only captured as a screenshot. The network and service findings in particular establish a genuinely clean, well-understood starting point: no exposed listening ports, one explainable DHCP connection, and a service list consisting entirely of standard Kali infrastructure.

Beyond simply running the required commands, this exercise demonstrated the verification discipline a baseline actually depends on. A command error was caught and corrected by checking file contents rather than assuming success, and every command was chosen and explained for a specific reason, not just executed because the task listed it.

These are exactly the habits that turn a folder of command output into evidence that is genuinely usable during a real future investigation, rather than a collection of screenshots someone would have to take on faith.